

K L Deemed to be University

Department of Computer Science and Engineering | 2026–2027, Odd Semester

Database Systems Engineering and Distributed Backend Development (25CS1302E)

Skilling Session 6: MongoDB – Aggregation and Design Patterns

Name	Chakrika Panakala	Roll No.	2520090042
Course	25CS1302E	CO Mapping	CO2
Session	Skilling 6	Date	

Aim

To build aggregation pipelines with `$match`, `$group`, `$project`, `$sort`, `$limit`, `$lookup` (the equivalent of a SQL JOIN) and `$unwind`, and to implement the Polymorphic, Bucket, Attribute, Computed and Outlier schema design patterns for Task Hub.

1. MongoDB Aggregation Pipeline

The Task Hub database is prepared with users and tasks. Aggregation pipelines are then used to summarize task effort, reshape documents, analyze tags, join tasks with users and generate a team workload report.

\$match + \$group + \$sort: Hours per Project

```
db.tasks.aggregate([
  {"$match": {"status": {"$ne": "TODO"}}},
  {"$group": {
    "_id": "$project",
    "tasks": {"$sum": 1},
    "total_hours": {"$sum": "$hours"},
    "avg_hours": {"$avg": "$hours"}
  }},
  {"$sort": {"total_hours": -1}}
])
```

The pipeline excludes TODO tasks, groups the remaining tasks by project, calculates task count, total hours and average hours, and sorts by total hours in descending order.

\$project: Computed Fields

```
db.tasks.aggregate([
  {"$match": {"project": "Backend"}},
  {"$project": {
    "_id": 0,
    "title": 1,
    "hours": 1,
    "is_done": {"$eq": ["$status", "DONE"]},
    "tag_count": {"$size": "$tags"},
    "effort": {
      "$cond": [{"$gte": ["$hours", 8]}, "large", "small"]
    }
  }}
])
```

`$project` reshapes the documents and computes `is_done`, `tag_count` and `effort` fields.

\$unwind + \$group + \$sort + \$limit: Top Tags

```
db.tasks.aggregate([
  {"$unwind": "$tags"},
  {"$group": {"_id": "$tags", "count": {"$sum": 1}}},
  {"$sort": {"count": -1, "_id": 1}},
  {"$limit": 3}
])
```

`$unwind` converts each tag array element into a separate pipeline document. The tags are grouped, counted, sorted and limited to the top three.

\$lookup + \$unwind + \$project: Task-Assignee JOIN

```
db.tasks.aggregate([
  {"$match": {"status": "IN_PROGRESS"}},
  {"$lookup": {
    "from": "users",
    "localField": "assignee_id",
    "foreignField": "_id",
  }}
])
```

```
      "as": "assignee"
    }},
    {"$unwind": "$assignee"},
    {"$project": {
      "_id": 0,
      "title": 1,
      "assignee": "$assignee.name",
      "team": "$assignee.team"
    }}
  ])
```

\$lookup performs the equivalent of a SQL LEFT OUTER JOIN between tasks and users. \$unwind converts the matched assignee array into an object.

2. Team Workload Report

```
db.tasks.aggregate([
  {"$lookup": {
    "from": "users",
    "localField": "assignee_id",
    "foreignField": "_id",
    "as": "u"
  }},
  {"$unwind": "$u"},
  {"$group": {
    "_id": "$u.team",
    "members": {"$addToSet": "$u.name"},
    "open_tasks": {
      "$sum": {"$cond": [{"$ne": ["$status", "DONE"]}, 1, 0]}
    },
    "hours": {"$sum": "$hours"}
  }},
  {"$sort": {"hours": -1}},
  {"$project": {
    "_id": 0,
    "team": "$_id",
    "hours": 1,
    "open_tasks": 1,
    "members": 1
  }}
])
```

This pipeline joins tasks with users, groups them by team, collects team members, counts open tasks and totals hours, then sorts teams by workload.

3. Schema Design Patterns

The second part of the session demonstrates five MongoDB schema design patterns for Task Hub: Polymorphic, Bucket, Attribute, Computed and Outlier.

Polymorphic Pattern

Documents with different shapes are stored in one collection and distinguished using a type field. The activities collection contains comment, status_change and file_upload documents.

```
db.activities.insert_many([
  {"type": "comment", "task_id": 103, "user_id": 1,
   "text": "Started GET /tasks"},
  {"type": "status_change", "task_id": 103, "user_id": 1,
   "from": "TODO", "to": "IN_PROGRESS"},
  {"type": "file_upload", "task_id": 101, "user_id": 1,
   "file_name": "er.png", "size_kb": 292}
])

db.activities.find({"task_id": 103}, {"_id": 0})
```

Bucket Pattern

Many small time-log entries are grouped into one document per user per day. Each bucket stores the entries, entry count and total minutes, with a maximum of 100 entries per bucket.

```
db.time_buckets.update_one(
  {"user_id": user_id, "day": "2026-10-01", "count": {"$lt": 100}},
  {"$push": {"entries": {"minutes": minutes}},
   "$inc": {"count": 1, "total_minutes": minutes}},
  upsert=True
)
```

Attribute Pattern

Varying product fields are stored as key/value pairs in an attrs array. One compound index on attrs.k and attrs.v supports queries across the varying attributes.

```
db.products.create_index([("attrs.k", ASCENDING), ("attrs.v", ASCENDING)])

db.products.find(
  {"attrs": {"$elemMatch": {"k": "seats", "v": {"$gt": 10}}}},
  {"_id": 0, "name": 1}
)
```

Computed Pattern

A pre-calculated summary is maintained on writes so that the application does not need to re-aggregate the data on every read. For the Backend project, task_count and total_hours are maintained.

```
db.projects.insert_one({
  "_id": "Backend",
```

```

        "task_count": 0,
        "total_hours": 0
    })

    for task in db.tasks.find({"project": "Backend"}):
        db.projects.update_one(
            {"_id": "Backend"},
            {"$inc": {
                "task_count": 1,
                "total_hours": task["hours"]
            }}
        )

```

Outlier Pattern

When a few documents are much larger than normal, the first N values are kept inline and the overflow is moved to a separate collection. The example keeps three watchers in the task and stores the remaining watchers in `watchers_overflow`.

```

WATCHER_LIMIT = 3
watchers = [f"user_{i}" for i in range(1, 8)]

db.tasks.update_one(
    {"_id": 105},
    {"$set": {
        "watchers": watchers[:WATCHER_LIMIT],
        "has_overflow": True
    }}
)

db.watchers_overflow.insert_one({
    "task_id": 105,
    "watchers": watchers[WATCHER_LIMIT:]
})

```

4. Key Output

Pipeline / Pattern	Observed Result
\$match + \$group + \$sort	Backend: 3 tasks, 26 hours, avg 8.6667; Frontend: 1 task, 9 hours
\$project	Computed is_done, tag_count and effort fields
Top 3 tags	api = 4, db = 2, ui = 2
\$lookup + \$unwind	Write FastAPI CRUD → Habeeb; Build dashboard page → Ananya Rao
Team workload	Backend: 28 hours, 2 open tasks; Frontend: 10 hours, 2 open tasks
Polymorphic	Different activity shapes stored in one collection
Bucket	User 1: 3 entries / 135 minutes; User 3: 1 entry / 50 minutes
Attribute	Task Hub Pro licence matched >10 seats
Computed	Backend: 4 tasks / 28 hours
Outlier	3 watchers inline; 4 watchers in overflow collection

Result

The aggregation pipelines produced hours per project, computed fields, the top three tags, task-assignee joins through \$lookup and a team workload report. The Polymorphic pattern stored three activity shapes in one collection, the Bucket pattern grouped time logs into daily documents, the Attribute pattern answered a key/value query with one compound index, the Computed pattern kept project totals up to date on write, and the Outlier pattern moved excess watchers into an overflow collection.